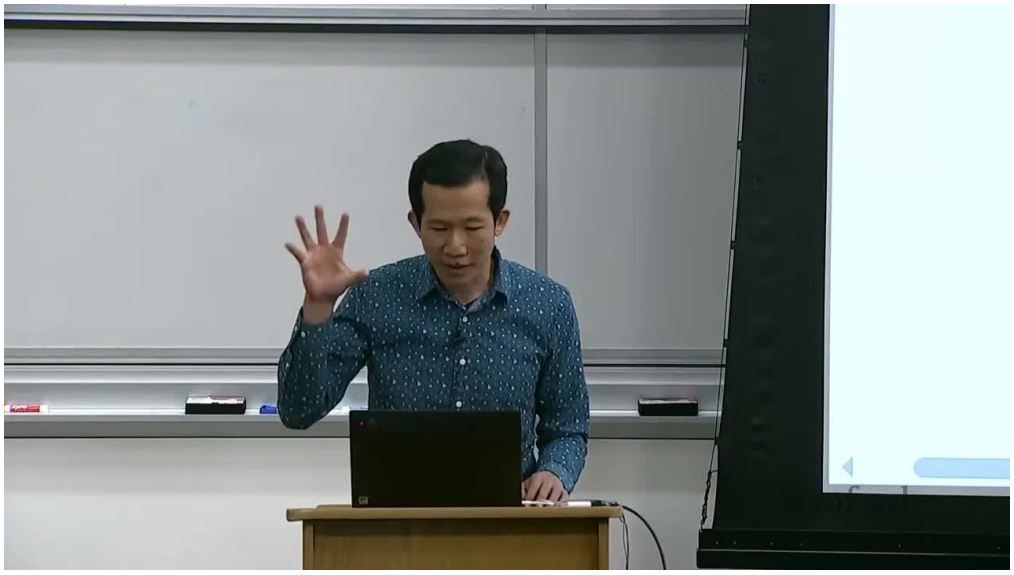
课程笔记

CS336 第 2 讲：PyTorch、资源核算与训练原语

Codex 基于 Stanford CS336 2025 第 2 讲视频、字幕与官方课程材料重写

2026-04-07



视频作者/频道：Stanford Online
发布日期：2025 年 04 月 10 日
视频时长：1:19:22
视频链接：https://www.youtube.com/watch?v=msHyYioAyNE

目录

1 这讲在整门课里扮演什么角色 **2**

1.1 为什么第 2 讲不再讲 Transformer，而改讲 primitives . . . 2

1.2 三个带走点：Mechanics、Mindset、Intuitions . . . 2

1.3 本章小结 . . . 3

2 资源核算为什么是语言模型工程的起点 **3**

2.1 Napkin math 不是粗糙，而是第一层必要近似 . . . 3

2.2 显存问题同样需要第一时间做数量级判断 . . . 3

2.3 为什么这门课要把内存和计算拆成两个账本 . . . 4

2.4 本章小结 . . . 4

3 Memory Accounting：tensor、精度与训练状态到底各占多少 **4**

3.1 为什么 tensor 是这门课最基础的对象 . . . 4

3.2 float32 是默认基线，但很贵 . . . 4

3.3 float16 为什么省内存，但又容易出事 . . . 5

3.4 bfloat16 为什么几乎成了现代训练的默认妥协 . . . 5

3.5 FP8：更便宜，但并不意味着可以无脑全量替换 . . . 5

3.6 Mixed Precision：为什么这其实是资源分层管理 . . . 6

3.7 本章小结 . . . 7

4 Compute Accounting：从 storage layout 到 FLOPs **7**

4.1 为什么 GPU 不只是更快的 CPU . . . 7

4.2 Tensor storage 与 stride：为什么“view 是免费的，copy 不是” . . . 8

4.3 Elementwise 与 MatMul：为什么矩阵乘法统治深度学习成本 . . . 8

4.4 FLOPs、FLOP/s 与硬件规格之间是什么关系 . . . 8

4.5 为什么前向 2、反向 4、总计 6 会成为一个反复出现的常数 . . . 9

4.6 MFU：为什么它是连接模型与硬件的桥 . . . 9

4.7 本章小结 . . . 9

5 Backward、模型对象与训练循环：从数学对象到工程系统 **9**

5.1 Backward 为什么大致比 forward 更贵 . . . 9

5.2 nn.Parameter、module 与 initialization 的角色 . . . 10

5.3 Data loading、pinned memory 与训练并行性 . . . 10

5.4 Optimizer：为什么 Adam 不只是“更新规则”，还是一笔大内存账 . . . 10

5.5 Checkpointing 与 mixed precision 为什么会在这讲收尾出现 . . . 10

5.6 本章小结 . . . 11

6 总结与延伸 **11**

1 这讲在整门课里扮演什么角色

第 1 讲建立的是世界观：为什么要从零构建语言模型，以及为什么效率和资源约束是理解整条系统链路的起点。第 2 讲则把这种世界观落到了第一个真正可执行的工程层面：**如果你要训练模型，究竟要先会哪些基础原语，又该如何对资源进行会计式管理？**

课程一开场就给出本讲的三条主线：第一，从 tensor 出发向上走到 model、optimizer 与 training loop；第二，在每个层次都问“内存和计算成本是多少”；第三，培养一种非常具体的研究工程直觉，即不要等模型跑炸了再问成本，而要在设计时就提前核算。

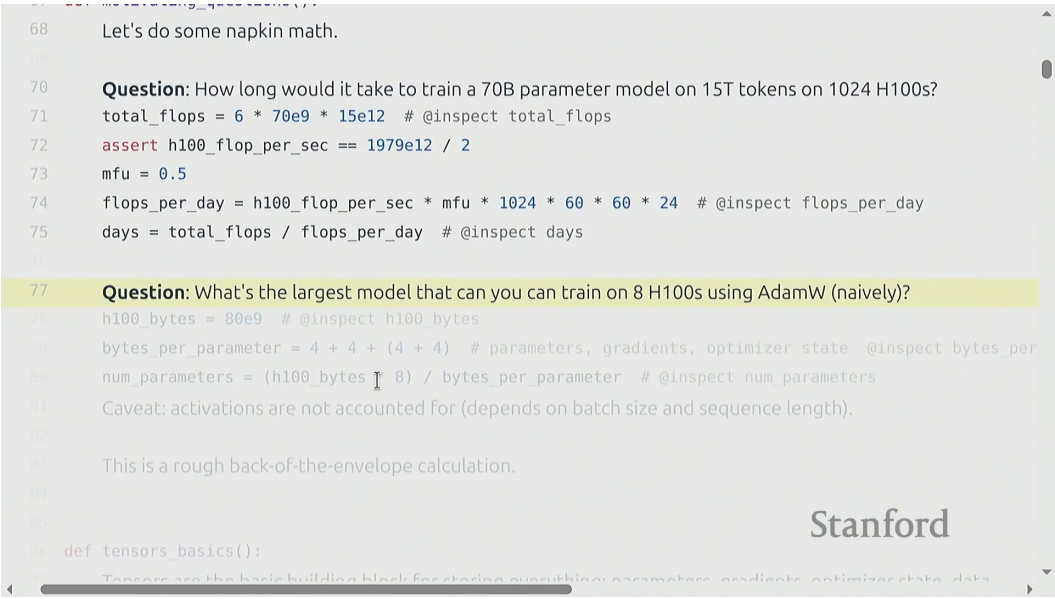


图 1: 课程视频里直接把两个 napkin math 问题写在可执行讲义界面上：训练 70B 模型需要多久，以及 8 块 H100 能容纳多大模型。[1]

这两个问题看起来只是热身题，实际上已经暴露了本讲的全部方法论：**训练不是先写网络再看运气，而是要先知道预算、瓶颈和数量级。**

1.1 为什么第 2 讲不再讲 Transformer，而改讲 primitives

课程明确说，这一讲不会再去复述 Transformer，因为外部已经有很多讲得很好的材料。它反而故意退一步，只讲“primitives needed to train a model”。这种取舍非常合理，因为 Transformer 的细节当然重要，但如果你连 tensor 如何占显存、矩阵乘法如何计 FLOPs、优化器状态如何额外吃内存、数据如何被 loader 送进 GPU 都不清楚，那么再复杂的架构也只能停留在纸面上。

第 2 讲真正训练的能力

这讲最重要的收获不是会不会背几个 PyTorch API，而是能不能把训练过程拆成一个个可核算的资源对象：参数、梯度、优化器状态、激活、数据搬运、矩阵乘法和通信。

1.2 三个带走点：Mechanics、Mindset、Intuitions

这讲沿用了第 1 讲对知识类型的划分，但重点进一步集中：

[1] 视频画面时间区间：00:02:22–00:02:40。该图来自课程视频中的屏幕画面。

- **Mechanics**：tensor、module、optimizer、train loop 这些对象在 PyTorch 里到底怎么工作。
- **Mindset**：resource accounting，要把显存、带宽、FLOPs 和 wall-clock time 绑定起来理解。
- **Intuitions**：这里只给 broad strokes，不在这讲里追求大模型层面的完整经验学。

这其实是在提醒读者：本讲不是“如何把一个现成模型调起来”的教程，而是“如何开始以大模型训练者的方式思考”的第一课。

1.3 本章小结

第 2 讲的定位非常清楚：它是从课程导论走向工程执行的过渡层。它把第 1 讲里“效率优先”的思想压缩成具体问题，并把后面关于 GPU、kernel、并行、scaling law 的内容提前预告成一套统一的资源核算框架。

2 资源核算为什么是语言模型工程的起点

2.1 Napkin math 不是粗糙，而是第一层必要近似

讲者开场的两个问题都属于 back-of-the-envelope calculation。很多人会误以为这种估算“不够精确，所以价值有限”，但大模型工程恰恰相反：在你还没有把数量级估清楚之前，所谓精细实现往往没有任何意义。

例如，“训练一个 70B 模型，在 15T tokens 上、用 1024 块 H100，需要多久?”这个问题逼你先把训练过程写成计算预算。课程采用的是一个极其经典但非常管用的一阶近似：每个 token 的训练总 FLOPs 大致与参数量成正比，整个训练总 FLOPs 近似为

$$\text{Total FLOPs} \approx 6 \times \text{参数量} \times \text{训练 token 数}$$

其中：

- 系数 6 来自后面会展开的一个经验事实：前向约 2 倍参数量 FLOPs，反向约 4 倍，总计约 6 倍。
- 参数量是模型规模的一阶代表。
- token 数是训练数据预算的直接体现。

这个近似当然不覆盖所有细节，例如通信、激活重算、kernel 实现差异、数据加载和实际 MFU，但它足以决定很多“到底值不值得做”的早期判断。

2.2 显存问题同样需要第一时间做数量级判断

第二个问题更直接地体现了内存核算的重要性：“8 块 H100 用 AdamW，天真地训练，最多能塞进多大模型?”课程给出的方法也很简单：按“每个参数要配套几份状态”来记账。

如果最朴素地用 float32 存参数和梯度，再加上 AdamW 的一阶、二阶优化器状态，那么每个参数背后至少会跟着多份额外存储。课程把它粗略拆成：

$$\text{bytes per parameter} \approx 4\ (\text{参数}) + 4\ (\text{梯度}) + 4 + 4\ (\text{优化器状态})$$

这会立刻让你意识到一个常被低估的事实：**模型参数本身从来不是显存唯一消费者**。在训练阶段，梯度、master weights、optimizer state 往往和参数本体一样重要，甚至更贵。

关于 napkin math 的两个重要 caveat

课程特别强调了两个 caveat。第一，参数和梯度并不一定要一直用 float32，实际中可能会用 bf16 并保留 float32 master copy。第二，这个粗算没有把 activations 算进去，而 activation 的成本又强烈依赖 batch size 与 sequence length。也就是说，粗算可以帮助你做一阶筛选，但绝不能把它当成完整答案。

2.3 为什么这门课要把内存和计算拆成两个账本

这一讲从一开始就把资源分成两大类：**Memory** 和 **Compute**。这个拆法非常关键，因为很多设计在其中一个账本上看似更优，却会在另一个账本上付出代价。

例如：

- 更高精度的数据类型通常更稳定，但会吃掉更多显存和带宽。
- 更激进的低精度格式能节省内存并提高吞吐，但也可能让训练变得不稳定。
- 某个操作看起来 FLOPs 不多，但如果频繁造成内存复制，仍然可能拖慢系统。

所以资源核算不是只看一个数字，而是同时盯住两个账本，并且不断问：**当前瓶颈到底是内存、带宽还是算力？**

2.4 本章小结

这一节建立了第 2 讲的基本方法：先粗算，再细化；先定数量级，再定实现细节。对于语言模型工程来说，资源核算并不是部署阶段的补充工作，而是决定你能不能顺利训练、是否值得训练、以及接下来应该优化哪一层的起点。

3 Memory Accounting：tensor、精度与训练状态到底各占多少

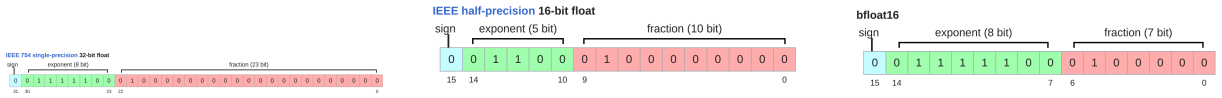
3.1 为什么 tensor 是这门课最基础的对象

课程强调，训练系统里几乎一切都最终以 tensor 形式存在：参数是 tensor，梯度是 tensor，optimizer state 是 tensor，输入 batch 是 tensor，激活也是 tensor。也就是说，**理解 tensor 怎么占内存、以什么数据类型保存、会不会共享底层 storage，本质上是在理解整个训练系统的存储基础**。

这就是为什么讲者宁愿从 `torch.zeros`、`torch.empty` 和 element size 开始，而不是立刻讲高层模型结构。因为如果你不理解这些最底层的表示形式，就很难真正把训练成本拆开。

3.2 float32 是默认基线，但很贵

课程首先从 float32 开始，因为这是 PyTorch 默认最容易理解的基线精度。在 scientific computing 里，float32 长期是标准做法；但在大模型世界里，如果所有参数、梯度、激活都一直坚持 float32，那么内存压力通常会非常大。



(a) float32 (b) float16 (c) bfloat16

图 2: 第 2 讲把三种常见浮点格式并列讲解：精度格式的选择，直接决定显存占用和数值稳定性的权衡。

课程还给了一个非常具有冲击力的例子：GPT-3 前馈层里仅一张大矩阵，用 float32 存储时就可以达到 GB 级别。这不是为了吓唬学生，而是为了让人意识到：**模型变大以后，连“单个矩阵”都足以成为你设计决策的对象**。

3.3 float16 为什么省内存，但又容易出事

float16 的诱惑很简单：每个元素只占 2 字节，立刻把很多 tensor 的存储减半。但问题同样明显：动态范围更差，尤其是对很小数值更敏感，容易 underflow。这一点在训练时很危险，因为梯度、归一化统计量、更新量里恰恰经常包含很多小数。

课程并没有停留在抽象结论，而是直接通过数值例子强调：某些在 float32 下还能被表示的小量，在 float16 下可能直接变成 0。于是我们得到一个非常重要的工程直觉：**低精度不是“白赚”，它节省的是内存和算力，代价是更脆弱的数值行为**。

3.4 bfloat16 为什么几乎成了现代训练的默认妥协

bfloat16 的设计思路就是保留接近 float32 的动态范围，同时仍然只占 2 字节。对深度学习而言，这往往是一个非常好的折中，因为训练里比起“极高尾数精度”，我们很多时候更需要“不轻易把小量压成 0”。

为什么 bfloat16 对大模型特别重要

训练稳定性通常不是由某一个超大数值毁掉，而更常出现在大量中小量的累积误差、归一化统计、梯度尺度和更新尺度上。bfloat16 保留了更大的指数范围，因此在现代训练里非常常见。

3.5 FP8：更便宜，但并不意味着可以无脑全量替换

FP8 的出现，是本讲在数据类型层面最前沿的一部分。课程指出，H100 支持两种 FP8 变体：E4M3 与 E5M2。直观上，这意味着进一步降低内存与数据搬运成本、提高吞吐成为可能，但也把训练稳定性问题推到了更前面。

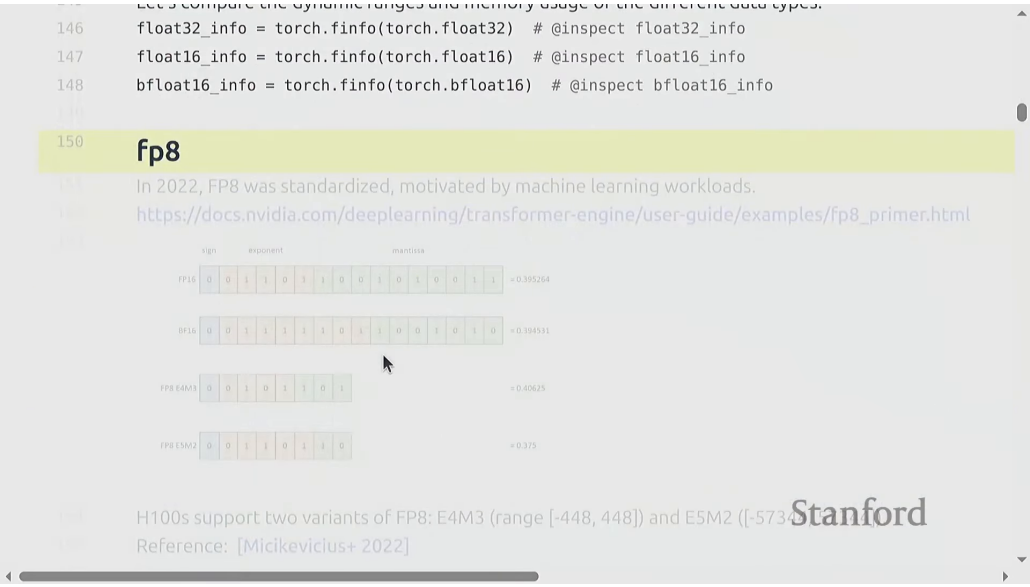


图 3: 课程视频里的 FP8 段落，重点并不是“FP8 一定更好”，而是它把内存节省与数值风险同时推向了更极端的地方。[2]

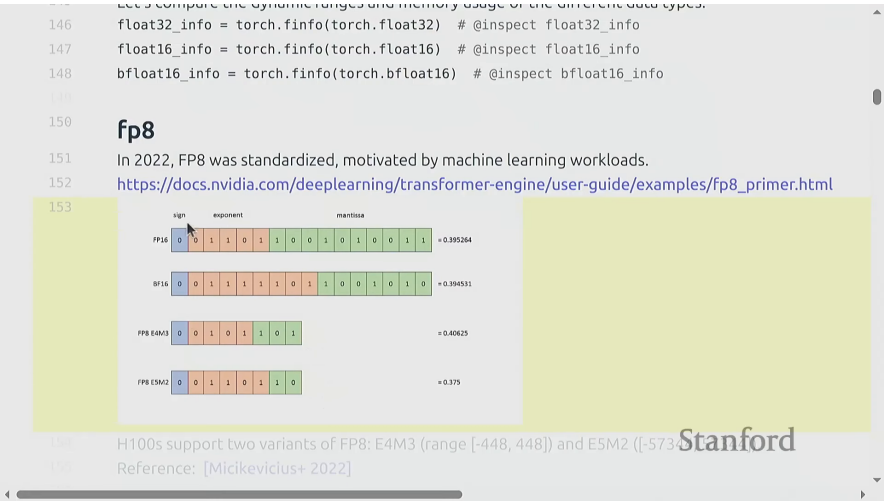


图 4: 课程视频里的 FP8 内容页，除了标题，还能看到对不同 FP8 变体的直观位宽拆分示意。[3]

课程给出的结论非常稳健：float32 当然可靠，但很贵；fp16/bfloat16/FP8 都能节省资源，但越往低精度走，训练风险越高。真正可行的方案不是简单“全面换成最低精度”，而是做混合精度分工。

3.6 Mixed Precision：为什么这其实是资源分层管理

课程把 mixed precision 解释得非常工程化：用低精度跑前向和部分线性层，是为了节省带宽和算力；保留更高精度的参数、梯度或 master copy，是为了守住稳定性底线。也就是说，mixed precision 的本质不是单个技巧，而是一种**按对象重要性给精度分层**的资源管理策略。

[2] 视频画面时间区间：00:11:30–00:11:41。该图来自课程视频中的屏幕画面。

[3] 视频画面时间区间：00:11:30–00:11:41。该图来自课程视频中的屏幕画面。

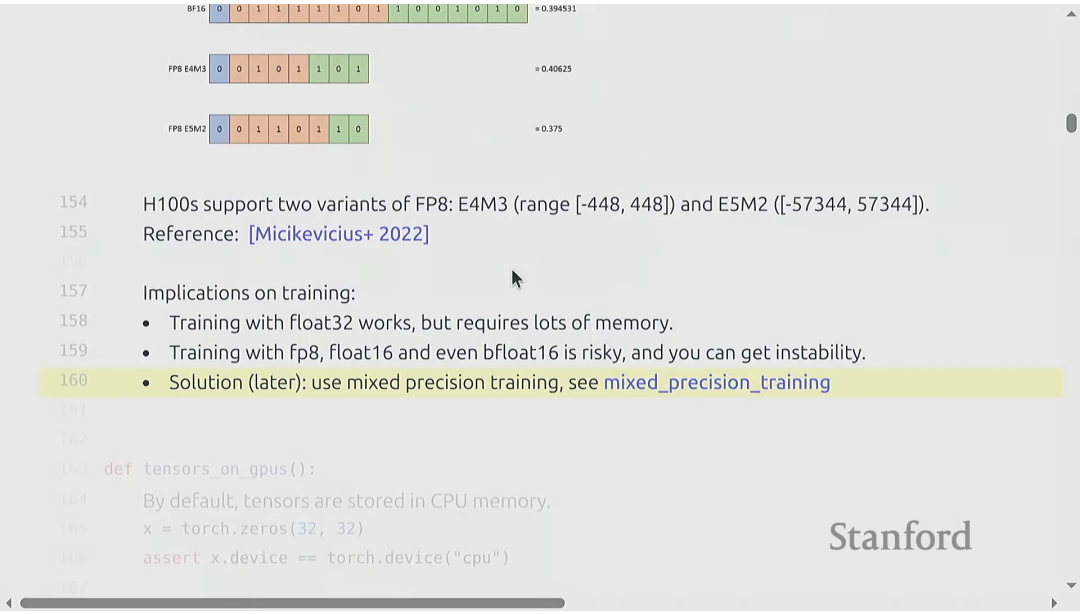


图 5: 课程视频在 FP8 段落后立刻把 mixed precision 拉出来，说明低精度的真正落点不是“全面替换”，而是精度分层。[4]

3.7 本章小结

这一节真正建立的是“内存账本”的意识。参数、梯度、优化器状态和激活都要占空间；数据类型并不只是数值分析问题，而是训练预算问题。第 2 讲用 float32、fp16、bfloat16 和 FP8 的对比，把这件事讲得非常具体：你每省一字节，通常都要在稳定性上重新做权衡。

4 Compute Accounting：从 storage layout 到 FLOPs

4.1 为什么 GPU 不只是更快的 CPU

课程先提醒一个基础事实：tensor 默认在 CPU 内存里，而真正想利用大规模并行计算，就必须把它搬到 GPU。这里的重点不只是“把 tensor 挪到设备上”，而是理解 GPU 的优势来自并行算力，而搬运本身也有代价。也就是说，**计算快不快，不只取决于乘法加法本身，还取决于你如何组织数据和访存**。

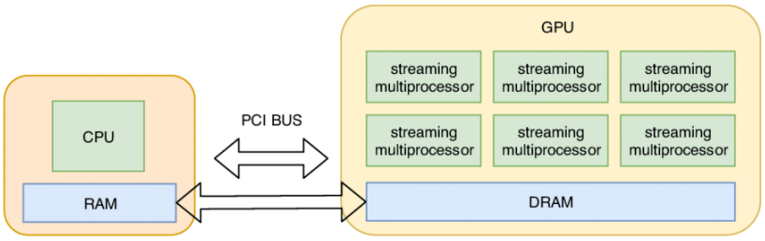


图 6: 课程用 CPU / GPU 对比提醒学生：并行算力只有在数据真正被送入 GPU 并被高效组织时才会兑现。

[4] 视频画面时间区间：00:13:04–00:13:16。该图来自课程视频中的屏幕画面。

这就是为什么后面 GPU、kernel、parallelism 那几讲会显得理所当然。第 2 讲已经把种子埋下了：如果数据搬运和布局不合理，你在高层架构上再漂亮，也很难跑出好效率。

4.2 Tensor storage 与 stride：为什么“view 是免费的，copy 不是”

这一讲很重要的一段，是把 tensor 解释成“指向一块已分配内存的指针 + 元数据”。元数据里最关键的一个概念就是 stride：每沿着某个维度移动一步，底层 storage 该跳过多少元素。

课程借此讲清楚一个很核心的工程事实：切片、view、transpose 等操作常常只是改变“如何解释同一块内存”，因此不一定会复制数据；而一旦触发显式复制，内存和计算成本都上来了。这个知识点看似很底层，却直接决定你能否正确理解后面更复杂的 kernel 和并行开销。对于一个二维 tensor 来说，所谓 stride，本质上就是“沿某个维度前进一步，要在底层 storage 中跳过多少元素”。一旦你明白这一点，就会自然理解为什么某些转置后的 tensor 不能直接再 ‘view‘，以及为什么有时必须先 ‘contiguous()‘。

关于 view 的一个常见误区

很多人看到高层 PyTorch API，会误以为“张量形状变了 = 开销很大”。课程专门提醒：只要底层 storage 没变、只是解释方式变了，view 往往几乎免费；真正昂贵的是复制和不必要的非连续重排。

4.3 Elementwise 与 MatMul：为什么矩阵乘法统治深度学习成本

在讲完 storage 之后，课程迅速把重心转到计算强度更高的操作上。它先承认 elementwise 运算当然很多，但随后明确指出：对于足够大的模型与张量，**几乎没有什么操作比大矩阵乘法更贵**。这就是为什么后面的 FLOPs 估算，最终大多都会落回 matmul。

在 batch 与 sequence 场景里，课程还强调了维度语义的重要性。你不只是做一堆抽象线性代数，而是在“对每个 batch 中的每个 token”重复相似结构。这也说明为什么维度命名、einops 和 tensor bookkeeping 会如此重要：一旦维度含义乱掉，工程实现很容易出错。

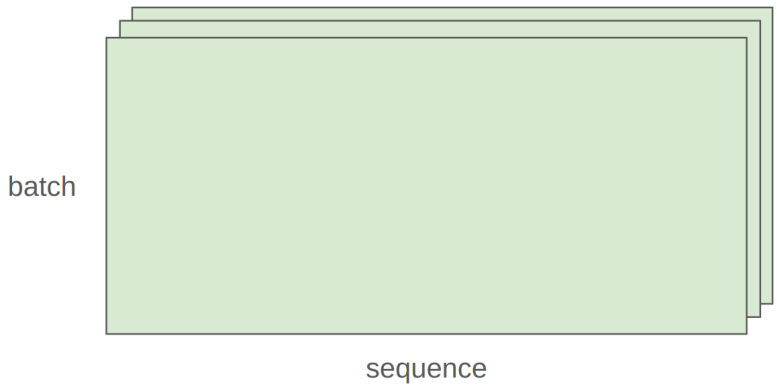


图 7: batch 与 sequence 维度不是装饰信息，而是后续张量操作与计算开销的核心语义。

4.4 FLOPs、FLOP/s 与硬件规格之间是什么关系

课程在这一段刻意区分了两个常常被混淆的词：

- **FLOPs**：一次训练或一次前向/反向总共做了多少浮点运算。

- **FLOP/s**：硬件每秒能做多少浮点运算，是速度上限。

这个区分非常重要，因为你不能把“总工作量”和“机器速度”混成一个概念。训练一旦被写成 FLOPs，你才能再拿硬件峰值与实际时间去比较，并进一步谈 MFU。

4.5 为什么前向 2、反向 4、总计 6 会成为一个反复出现的常数

课程用线性模型示例解释了一个在大模型训练里极其常用的一阶经验：前向大致需要 2 倍“数据点数 × 参数数”的 FLOPs；反向大致再来 4 倍；于是总计约为 6 倍。这当然不是对所有网络的精确闭式，但对以 matmul 为主的模型来说，它提供了一个足够好用的估算尺度。

为什么“6 倍规则”很有用

一旦你接受这个一阶近似，训练成本就能被迅速压缩成“数据规模 × 参数规模”的乘法问题。很多是否值得尝试的训练方案，先用这个尺度就可以筛掉一大批不现实选择。

4.6 MFU：为什么它是连接模型与硬件的桥

课程把 MFU 定义为：

$$\text{MFU} = \frac{\text{模型实际完成的 FLOP/s}}{\text{硬件承诺的峰值 FLOP/s}}$$

其中：

- 分子是你训练时真正兑现出来的有效计算吞吐。
- 分母是规格表上的理论峰值。
- MFU 越高，说明你的实现越接近把硬件“喂饱”。

课程还特别提醒，MFU 不是为了炫耀数字，而是为了判断瓶颈到底在哪里。如果峰值很高、实际很低，问题可能在 kernel、data loader、访存、调度，或者低效的小矩阵上，而不在模型“理论上算得不够多”。

4.7 本章小结

Compute accounting 这一节把第 2 讲从“内存会计”推进到“性能会计”。storage layout、view、stride、matmul、FLOPs 和 MFU 被串成了一条链：你先知道数据怎么摆，再知道主要计算发生在哪里，最后才知道硬件到底有没有被真正利用起来。

5 Backward、模型对象与训练循环：从数学对象到工程系统

5.1 Backward 为什么大致比 forward 更贵

课程在讲梯度时没有从复杂自动求导原理讲起，而是直接用线性模型说明：对于很多矩阵乘法主导的网络，反向传播本质上还是在做与前向规模相当、甚至更多的矩阵运算。因此，反向 4 倍、前向 2 倍、总共 6 倍，才会成为那么常见的工程近似。

这背后的直觉很重要：**训练不只是“前向再顺手求个导”，而是完整多了一层高成本计算**。所以任何能减少反向成本、减少状态开销或减少激活保存需求的技巧，都会在后续课程里变得格外有价值。

5.2 nn.Parameter、module 与 initialization 的角色

到了 model 层，课程重点没有放在“深奥架构”，而是放在模型对象最根本的工程定义：参数在 PyTorch 中就是 `nn.Parameter`。这听上去平淡，但它意味着参数是否需要梯度、如何被 module 收集、如何被 optimizer 跟踪，都是框架语义的一部分。

课程还专门强调初始化。为什么？因为错误的初始化会让激活或梯度尺度随着输入维度迅速放大，训练一开始就不稳定。Xavier 一类初始化的核心思路，就是根据输入维度重新缩放，让不同层的尺度保持在比较合理的范围内。这一点在深层模型里尤其关键。

5.3 Data loading、pinned memory 与训练并行性

课程没有把 data loader 当成普通“输入管道细节”，而是把它视为训练系统的一部分。语言模型数据常常是极大规模 token 序列，不可能一次性完整载入显存，甚至不适合一次性载入主内存。因此 memmap、随机采样 batch 起点、以及 pinned memory 这类概念，都直接影响吞吐。

Pinned memory 之所以值得强调，是因为它让 CPU 到 GPU 的拷贝可以异步执行，从而有机会把“准备下一批数据”和“GPU 正在处理当前 batch”并行起来。也就是说，哪怕这一步本身看起来只是 I/O 细节，它依然会体现在整体训练效率上。

5.4 Optimizer：为什么 Adam 不只是“更新规则”，还是一笔大内存账

课程从 AdaGrad、RMSProp、Adam 这些优化器的历史关系出发，逐步解释为什么现代训练默认常用 AdamW 一类方法。但更重要的点在于：优化器状态不是抽象概念，它会真实占显存。

例如 Adam 需要为每个参数维护额外的一阶、二阶统计量，这正是前面内存核算中“每个参数不止一份状态”的来源。所以当你说“我要训练一个模型”时，你其实同时也在说“我要维护一个同样规模甚至多倍规模的状态系统”。

```python
for step in range(num_steps):
    x, y = next_batch()
    x = x.to(device, non_blocking=True)
    y = y.to(device, non_blocking=True)

    optimizer.zero_grad(set_to_none=True)
    logits = model(x)
    loss = criterion(logits, y)
    loss.backward()
    optimizer.step()
```

Listing 1: 极简训练循环伪代码：第 2 讲真正想让你理解的工程骨架

上面这段伪代码看起来很普通，但第 2 讲真正要你看到的是：每一行后面都有资源含义。拷贝是否异步、梯度何时释放、参数状态如何更新、loss backward 会产生多少额外计算，这些都和训练效率直接相关。

5.5 Checkpointing 与 mixed precision 为什么会在这讲收尾出现

课程在后半段把 checkpointing 和 mixed precision 放到一起，不是随意排列。它们都代表一类非常典型的工程思想：**用额外控制流或额外复杂度，换资源效率**。

- checkpointing 的意义是：训练很可能会长时间运行并发生故障，所以必须周期性保存模型和优化器状态。
- mixed precision 的意义是：不同对象并不需要同样精度，可以通过分层精度设计同时拿到更高吞吐与可接受稳定性。

它们都说明了一点：到了真实训练系统里，“模型定义”远远不是全部，鲁棒性和资源效率同样是第一等公民。

5.6 本章小结

这一节把第 2 讲从张量和 FLOPs 推到了训练系统本身。你开始看到：参数、梯度、优化器状态、batch、loader、pinned memory、checkpointing 和 mixed precision 其实共同构成了一个完整训练对象。也正因如此，第 2 讲才会成为后面所有更高级系统章节的基础。

6 总结与延伸

把第 2 讲压缩成一句话，就是：**训练语言模型不是先有一个模型、再把它扔给 GPU；而是先要把整个训练过程拆成可核算、可搬运、可存储、可恢复、可优化的资源对象**。

如果只记住四个结论，我建议是：

1. 先做数量级核算，再谈实现细节。
2. 内存账本至少要同时盯住参数、梯度、优化器状态与激活。
3. 计算账本最终大多会回到 matmul、FLOPs 和 MFU。
4. 训练循环不是几行代码，而是一整套资源调度系统。

第 2 讲的压缩结论

这讲真正教会你的，不是某个 PyTorch 技巧，而是如何把“训练模型”翻译成一套资源会计语言。一旦会了这门语言，后面的 GPU、kernel、parallelism、inference 和 scaling law 才真正有了统一坐标系。

从后续学习路径看，第 2 讲之后最自然的三个延伸方向是：

- 用 GPU 和 kernel 视角进一步追问：既然 matmul 主导成本，为什么某些 kernel 仍然跑不满硬件？
- 用 parallelism 视角追问：当单卡放不下模型时，参数、梯度与状态到底怎么分片？
- 用 inference 视角追问：训练里这套会计方法到了 decode 阶段为什么会完全改写？